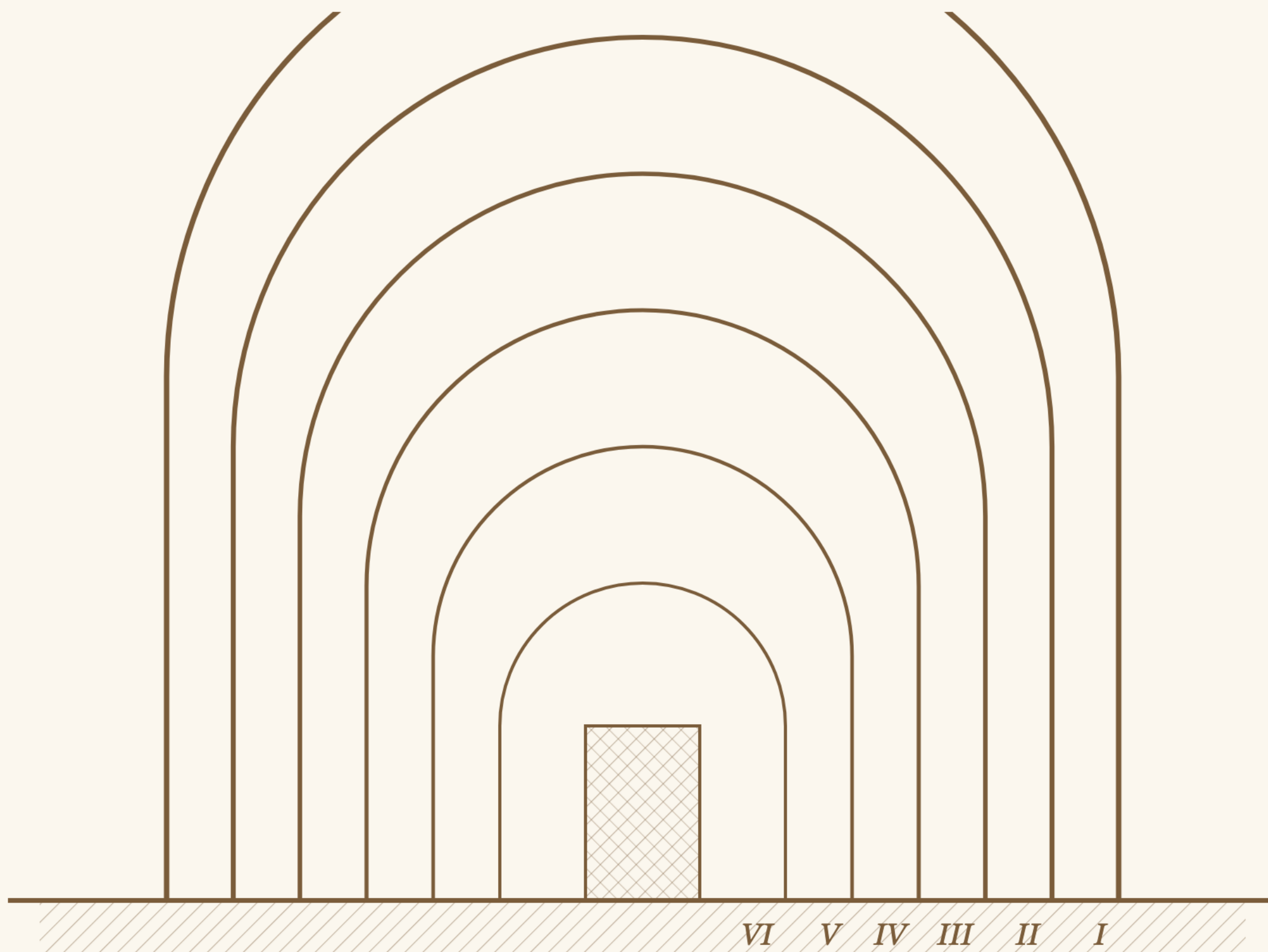


AGENTIC AI · ARCHITECTURE · PRACTICE

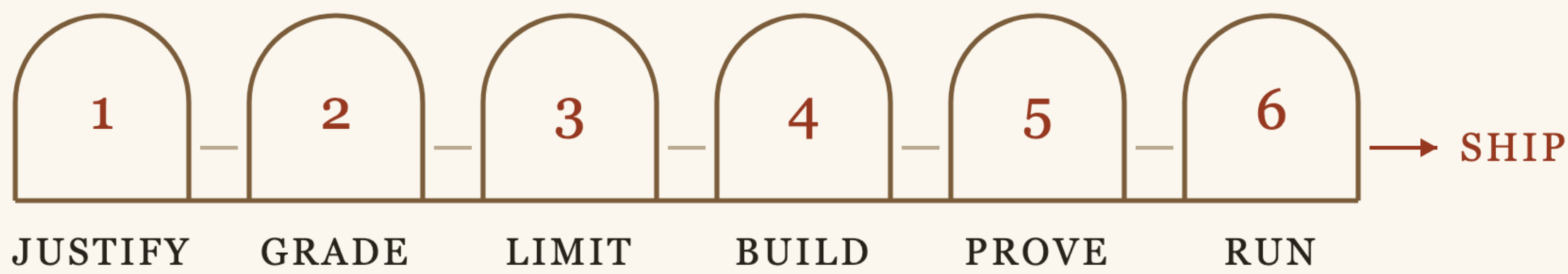


The Six Gates

*A practitioner method for deciding, bounding,
and operating AI agents*

BHARGAVA P

When someone says “we want to build an AI agent,” there is no TOGAF to reach for, no well-architected framework. This is the method I use instead: six gates, passed in order. Most agent failures trace back to a skipped gate.



<i>I</i>	Justify	<i>Should this be an agent at all?</i>
<i>II</i>	Grade	<i>How much can this thing hurt us?</i>
<i>III</i>	Limit	<i>Who is allowed to write what?</i>
<i>IV</i>	Build	<i>The smallest supervised loop that could work</i>
<i>V</i>	Prove	<i>What evidence buys this thing its freedom?</i>
<i>VI</i>	Run	<i>Who maintains the system that maintains itself?</i>

With a preface, a worked example, the paper trail, and the usual objections at the back.

Why gates

Clouds got a well-architected framework. Enterprises got TOGAF. Agents got a demo culture. When the industry is missing a method, every team improvises one, and improvised methods fail in the same six places: agents that should have been cron jobs, autonomy granted by enthusiasm, tool access nobody mapped, architectures built for the demo, evidence that amounts to “we watched it,” and systems nobody owns at 2am.

So this is not a framework. It is six questions, asked in a fixed order, each one a **gate**: you do not proceed until you pass. Any engineer can run them in a one-hour session with a whiteboard. The order matters, because every gate protects the ones behind it.

Read it in ten minutes. Use it on your next agent request.



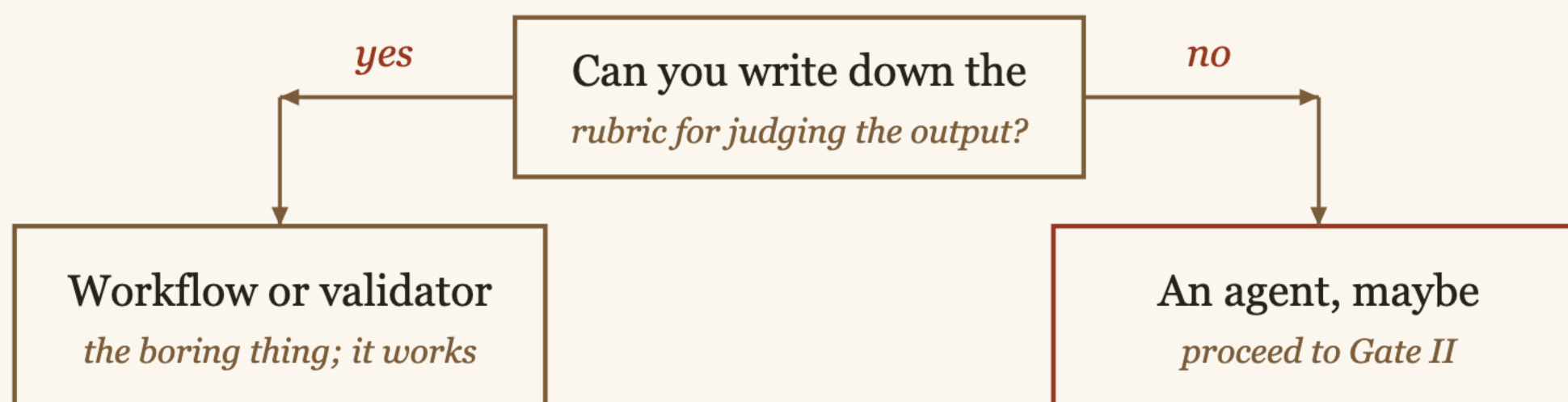
A method you can hold in your head is the only kind that survives a meeting.

Justify

Should this be an agent at all?

justify · *verb* · to prove a task needs an agent at all, before any architecture is drawn.

Apply the **Rubric Test**: can you write down the checklist for judging the output? If yes, build a workflow or a validator, not an agent. Agents earn their complexity only where judgment is needed at runtime, on inputs you cannot enumerate. To pass, complete the sentence: “This must be an agent because the decision cannot be specified in advance, specifically because...”



Agents earn their complexity only where judgment is needed at runtime.

Figure 1. The Rubric Test, applied before any architecture is drawn.

FAILURE SMELLS LIKE

A beautiful agent demo doing a task a cron job could do.



If you can write the checklist, build the boring thing. It works.

Grade

How much can this thing hurt us?

grade · *verb* · to place an agent on the risk ladder by how much damage its worst day can do.

Place the agent on the ladder before designing anything. The tier decides the evidence you will need, the audit trail you will keep, and what you build first. Tier 4 is where real money, real promises and case law live. Ask Air Canada how the chatbot tier went.

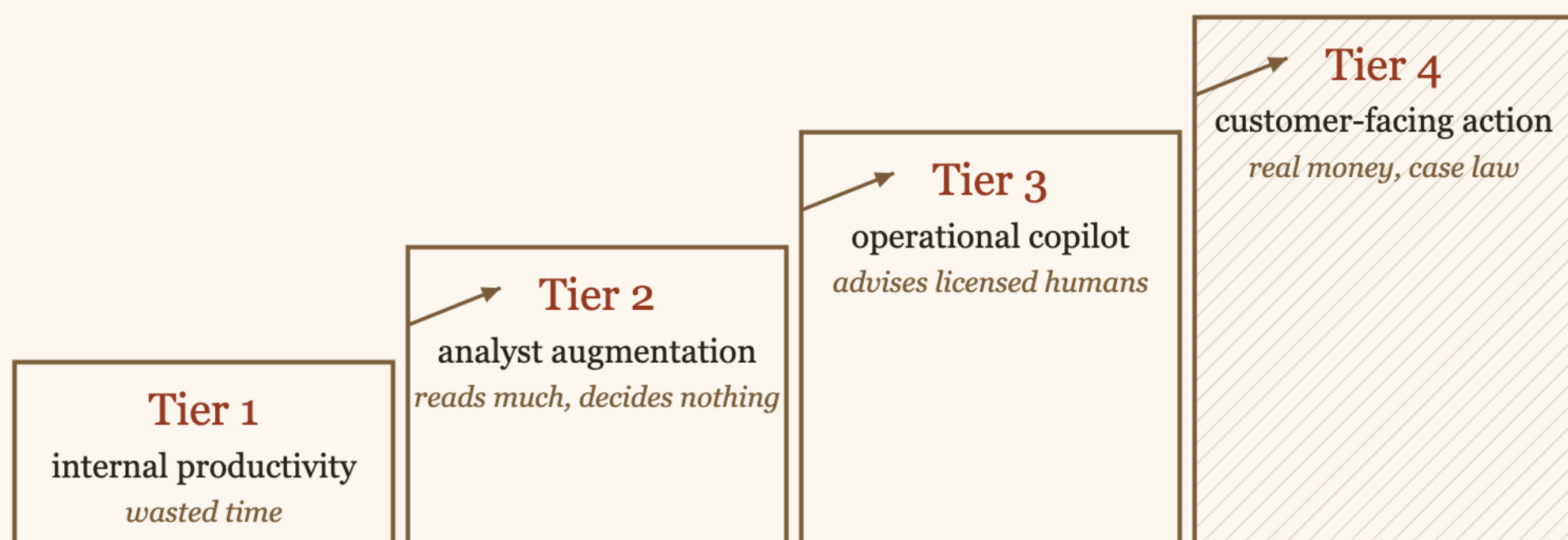


Figure 2. The four-tier risk ladder. The climb is earned, never assumed.

FAILURE SMELLS LIKE

Starting at Tier 4 because that is where the ROI slide points.



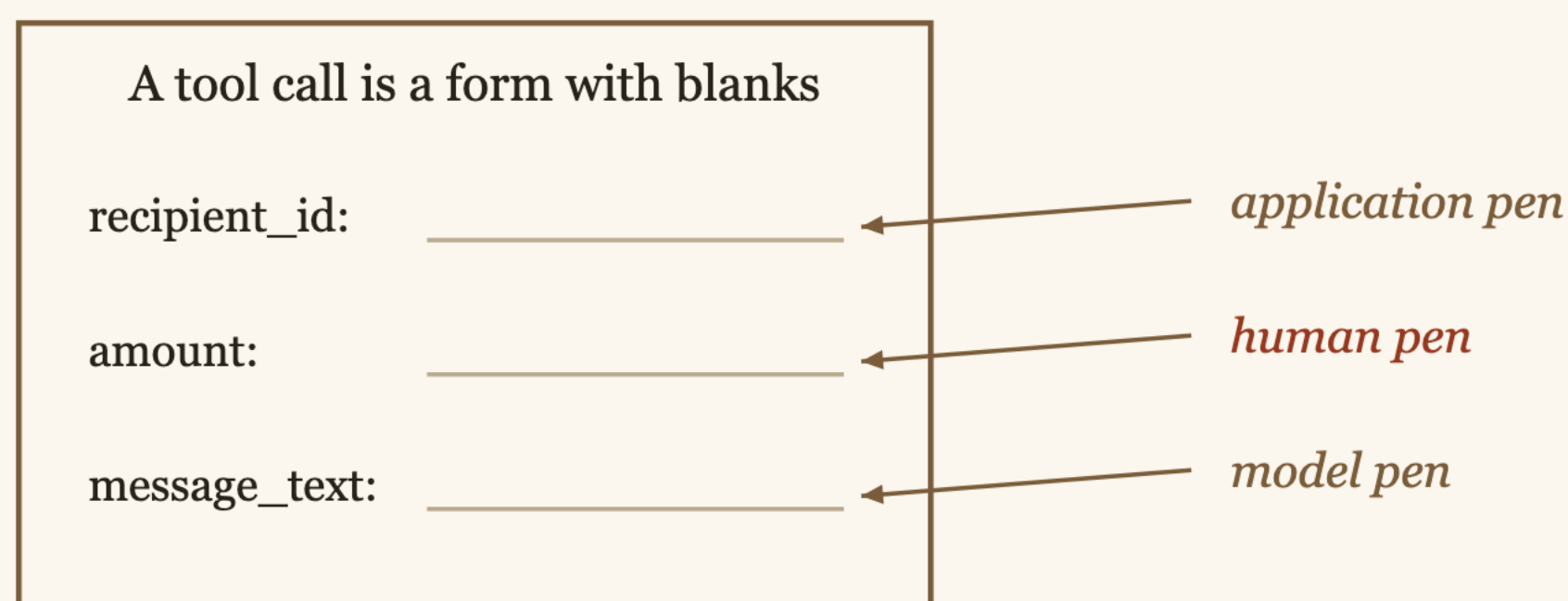
Start low. Earn the climb.

Limit

Who is allowed to write what?

limit · *verb* · to fix, argument by argument, what the agent may write, and enforce it outside the prompt.

A tool call is a form with blanks. For every blank, name who holds the pen: the **application** (IDs, amounts, recipients, injected by code), the **model** (free text, its actual job), or a **human** (money, waivers, precedent). Then enforce it at the tool boundary, deterministically. To pass: show the tool catalog with every argument labeled.



enforced at the tool boundary, deterministically

Figure 3. Three pens for one tool call. Prompts request; boundaries enforce.

FAILURE SMELLS LIKE

A consequential argument that is model-writable because “the prompt says to be careful.”



Least privilege was never just about tools. It is about arguments.

Build

What is the smallest supervised loop that could work?

build · *verb* · to assemble the smallest supervised loop that could do the job.

One agent before multi-agent. Supervised before autonomous: the agent drafts, a human approves, every consequential call passes a checkpoint. Underneath, a cascade: small models for rubric-writable steps, the frontier model only for judgment, humans for the truly weird.

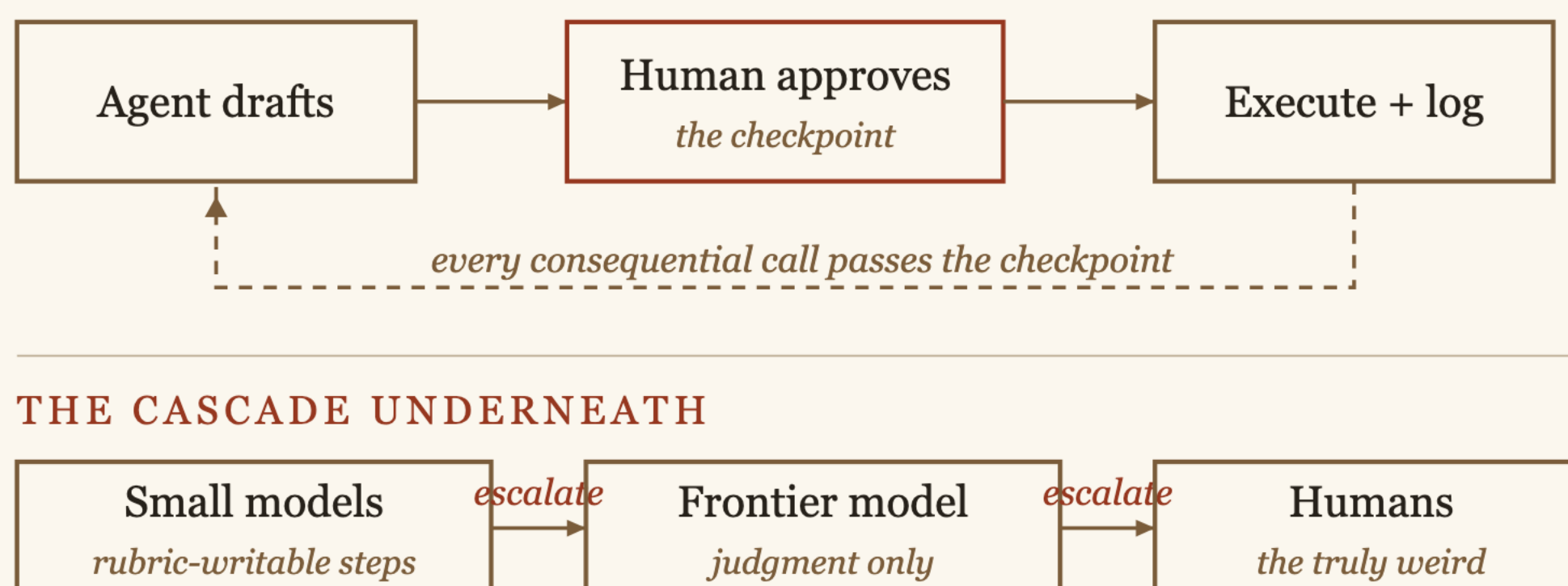


Figure 4. The supervised loop, and the model cascade underneath it.

FAILURE SMELLS LIKE

A five-agent orchestra on day one, fully autonomous, with no human checkpoint and no logs.



The best agent architecture usually invokes the expensive model least.

Prove

What evidence buys this thing its freedom?

prove · verb · to purchase autonomy with frozen, reproducible evals instead of demos.

Nothing gets autonomy because it demoed well. Freedom is purchased with evals: a benchmark passed before each promotion, from suggestion-only to human-approved to autonomous, tool by tool. Freeze the eval before the run. Keep the results. Let anyone reproduce them. And the gate swings both ways: any change to the model, the prompt, or the tools sends the agent back through it.



*a benchmark passed before each promotion, tool by tool,
never a demo that impressed the room*

Figure 5. Promotion is gated by frozen, reproducible evals, never by demos.

FAILURE SMELLS LIKE

“How do we know it works?” answered with “we watched it for a while.”



Autonomy is not granted. It is purchased with evidence.

Run

Who maintains the system that maintains itself?

run · *verb* · to keep a production agent owned, logged, and pruned, with a name on the pager.

Production agents rot in three places: prompts accrete, tool catalogs sprawl, and self-improving agents now write their own skills. The cure is a ledger: prompts and skills in version control with owners, provenance notes on anything the agent authors, a scheduled deletion pass, escalation logs reviewed like gold, and an off switch you have actually tested.

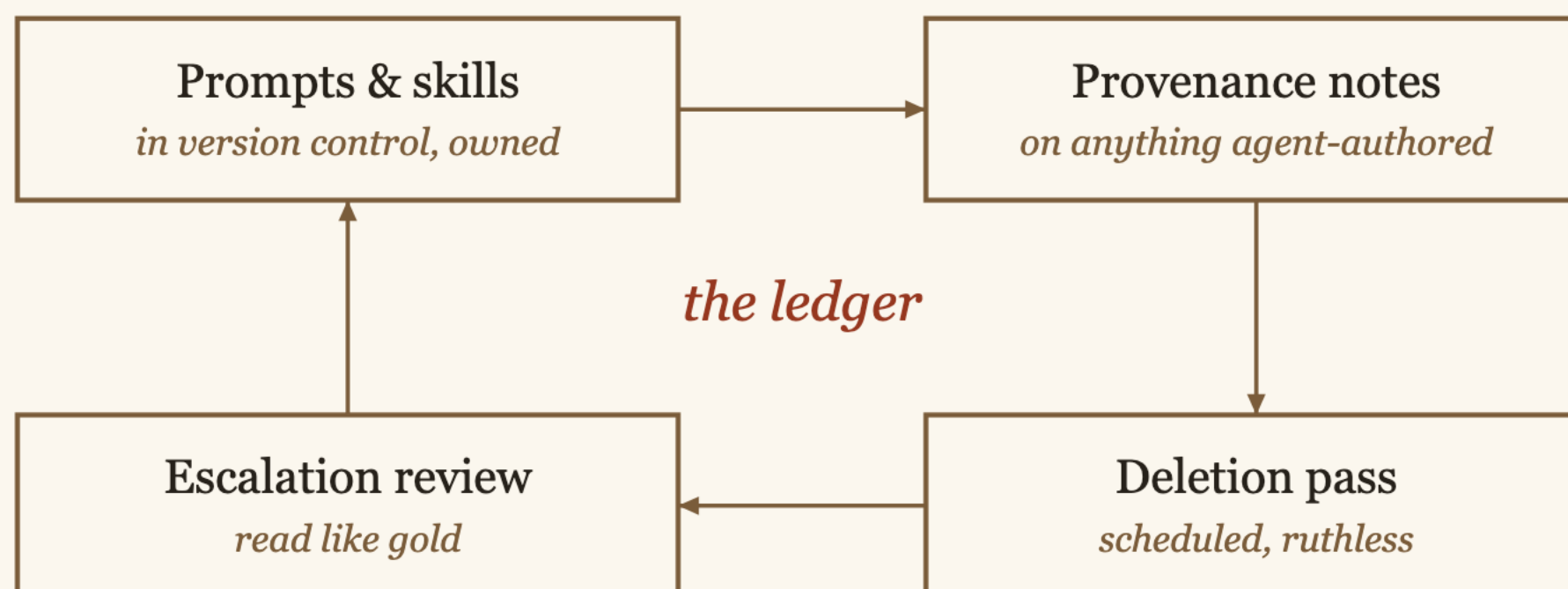


Figure 6. The operating ledger. Nothing enters the system without an owner.

FAILURE SMELLS LIKE

*A system prompt that has grown for six months and never shrunk once.
That is not a prompt. That is a landfill.*



Give the agent the pen. Keep the ledger.

One agent, six gates

An expense-report triage agent walks the whole path.

The request arrives the usual way: “can we get an AI agent to handle expense reports?” Here is the one-hour session, gate by gate.

<i>I</i>	Justify	Policy limits are a rubric, so they become a validator. The ambiguous twenty percent, handwritten receipts and split trips, cannot be enumerated. The agent earns its place there only.
<i>II</i>	Grade	It can block someone’s reimbursement, which is money-adjacent. Tier 2: it reads everything and decides nothing. It recommends.
<i>III</i>	Limit	approve_or_reject: human pen. employee_id and amount: application pen. The rationale text: model pen. Enforced in the tool layer, not the prompt.
<i>IV</i>	Build	One agent drafts a recommendation citing the policy lines it used. A small model sorts receipt types; the frontier model sees only the weird ones. A reviewer approves. Everything is logged.
<i>V</i>	Prove	Two hundred frozen historical cases. It must beat the checklist baseline before it may auto-approve even the cleanest category.
<i>VI</i>	Run	Prompt in version control with an owner. Monthly deletion pass. Escalations reviewed weekly. A name, not a team, on the pager.



Ten minutes per gate, and the design wrote itself.

What each gate leaves behind

A gate that leaves no artifact was not passed. It was waved.

Auditors, new team members, and your own future self should be able to reconstruct every decision from six small documents. None of them is longer than a page.

I

Justify

The completed sentence: “this must be an agent because...”
signed by product owner + architect

II

Grade

A grade memo with the worst-case failure written down
signed by the risk owner

III

Limit

The tool catalog with every argument’s pen labeled
signed by security

IV

Build

The architecture sketch and the escalation log location
signed by engineering

V

Prove

The frozen eval, its results, and how to reproduce them
signed by the eval owner

VI

Run

The ledger, and a named 2am owner
signed by operations



Evidence is the only deliverable that survives a reorg.

Yes, but

Three pushbacks arrive in every review. Here are the answers.

“This will slow us down.”

Gates cost hours. Incidents cost quarters. Gate I is one sentence, and it is the sentence that cancels the six-month project that should never have started.

“We’re just doing a pilot.”

Pilots become production by demo momentum, never by decision. Grade it anyway. A pilot is simply an agent standing at Gate V without evidence yet.

“Our vendor handles safety.”

Your vendor bounds the model. Nobody but you can bound your arguments, your tools, and your tiers. Gates II, III, and VI are yours alone, and they are the ones that make the news.



Gates are not bureaucracy. Bureaucracy is what grows where method is missing.

The agentic era does not need new virtues.
It needs the old ones applied *before the*
demo
instead of after the incident.

THE AUDIT QUESTION

*Your current agent project: which gate is it standing at,
and which one did it skip to get there?*

This document is the full method. Steal it whole.
The gates protect each other.

BHARGAVA P

Agentic AI architect. Writes about agents that survive production.

The Air Canada case

Moffatt v. Air Canada, British Columbia Civil Resolution Tribunal, February 2024. The tribunal rejected the airline’s argument that its chatbot was “a separate legal entity responsible for its own actions.”

On building agents

Anthropic, “Building Effective Agents” (2024): the case for simple, composable patterns over frameworks. OpenAI, “A Practical Guide to Building Agents” (2025).

On risk and governance

NIST AI Risk Management Framework 1.0 (2023) and ISO/IEC 42001:2023. The gates are a team-level, one-hour walk through the same terrain those frameworks cover at organization level.

Prior work

The Rubric Test (Gate I), the four-tier ladder (Gate II), and the three-pens model of argument authority (Gate III) are the author’s own frameworks, published on LinkedIn in 2026.

The Six Gates, version 1.0 · September 2026

Written, illustrated, and typeset by Bhargava P

Share it freely, with attribution. The gates protect each other.